

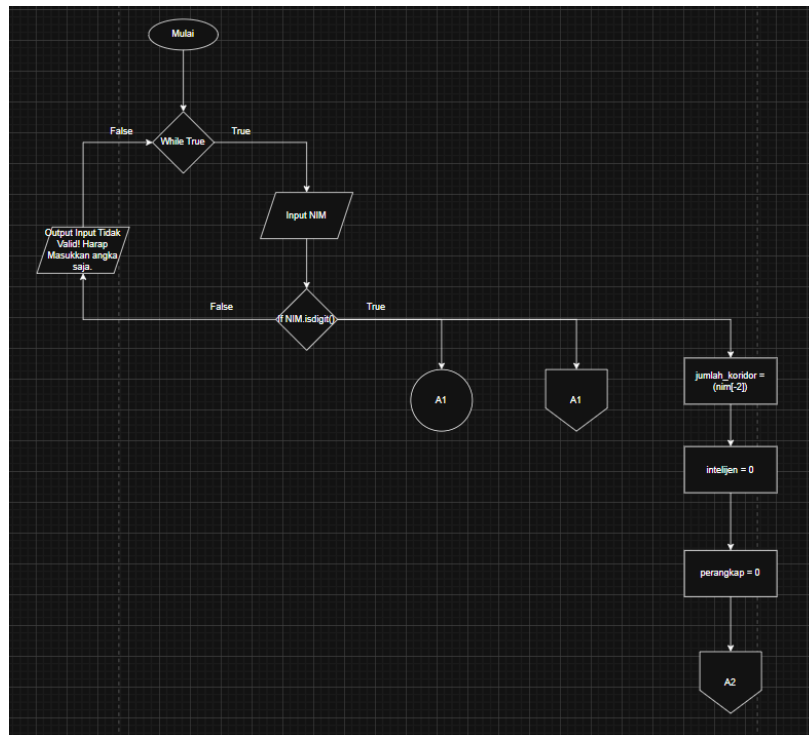
LAPORAN PRAKTIKUM
POSTTEST 4
ALGORITMA PEMROGRAMAN DASAR



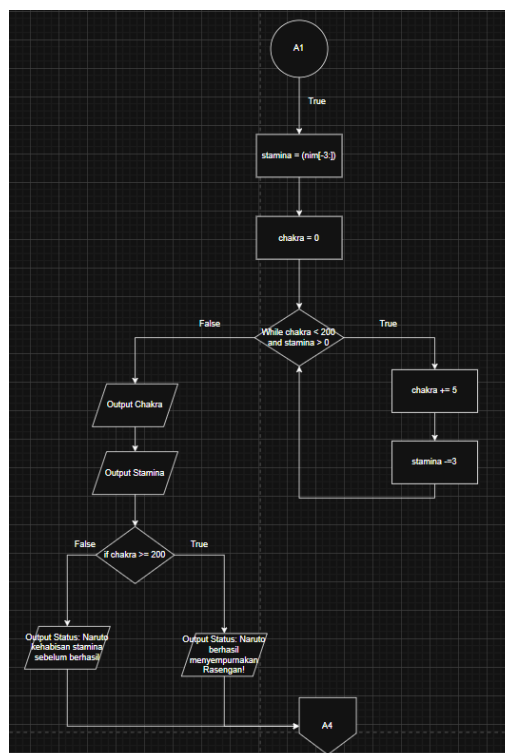
Disusun oleh:
Muhammad Fajar (2509106117)
Kelas (C2 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

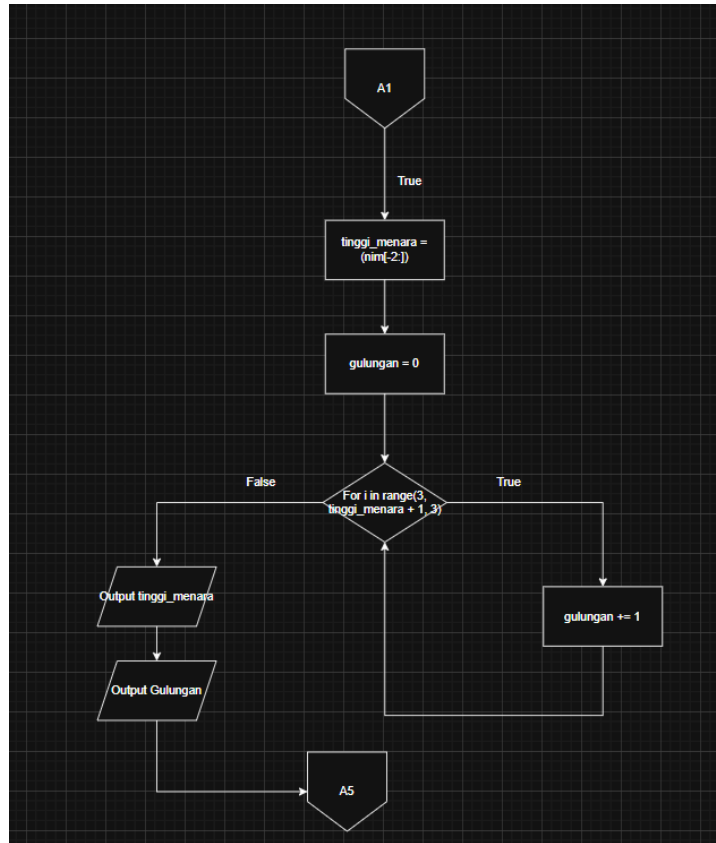
1. Flowchart



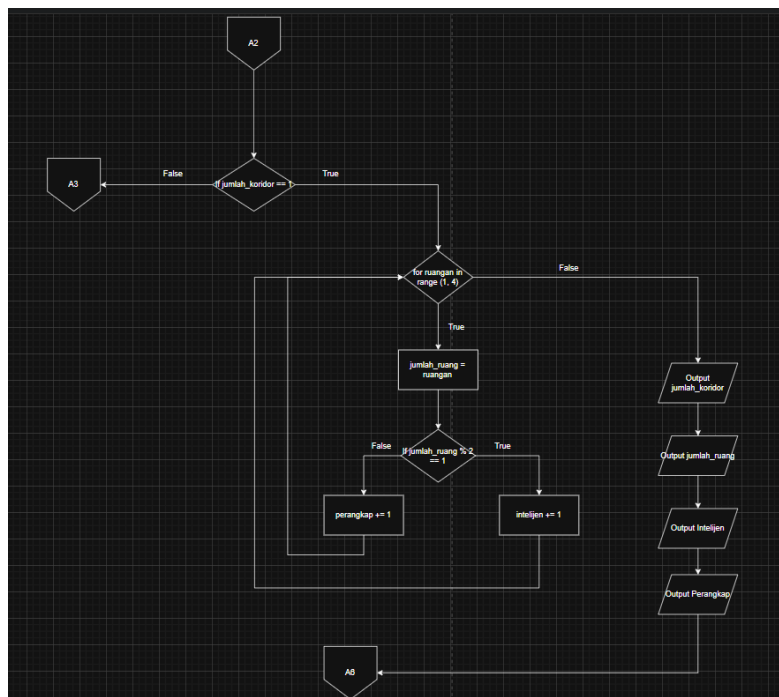
Gambar 1.1 Flowchart



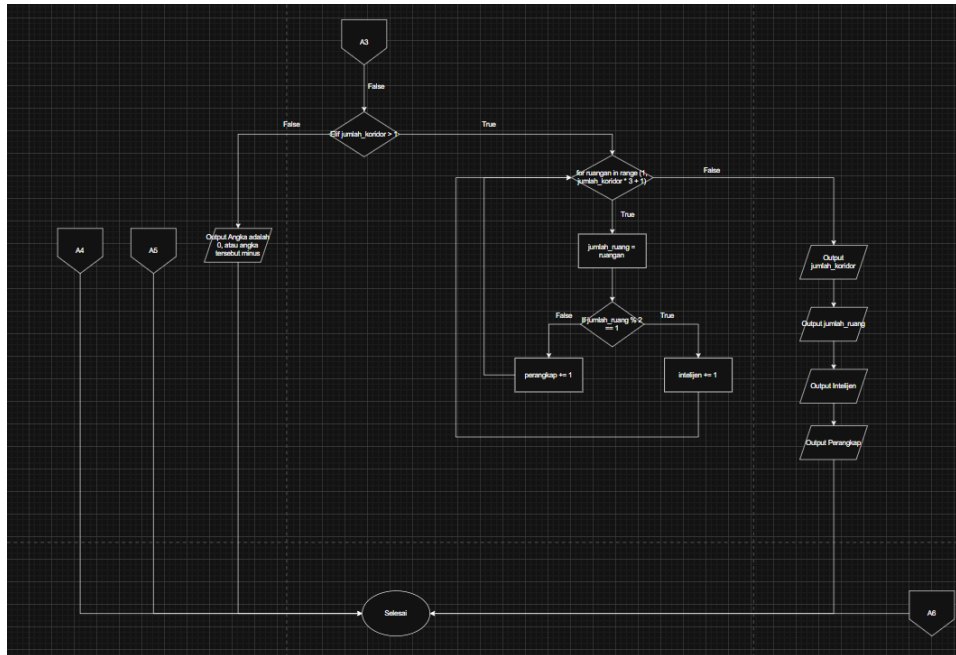
Gambar 1.2 Flowchart



Gambar 1.3 Flowchart



Gambar 1.4 Flowchart



Gambar 1.5 Flowchart

Penjelasan:

1. Mulai
2. Mengecek Apakah true
3. Input NIM
4. Jika Input NIM adalah huruf
5. Maka akan mengeluarkan pesan Output/Print (“Input tidak valid! Harap masukkan angka saja.”)
6. Maka akan kembali ke Nomor 2
7. Jika Input adalah angka
8. Maka akan masuk ke kondisi pertama
9. Stamina (nim[-3:]), yang di ambil nim saya bagian (3 digit di belakang)117
10. Lalu chakra = 0
11. Lalu While chakra < 200 and stamina > 0, ini maksudnya kalua chakra lebih kecil dari 200 dan stamina lebih besar dari 0
12. Maka chakra += 5, maksudnya chakra (0 +5 = 5),
13. Dan stamina -= 3, maksudnya stamina (117(misalnya)-3 = 114)
14. Maka akan kembali ke 11, sampai terpenuhi sama whilenya (hasilnya chakra 195 < 200 dan stamina 0 > 0), maka hasil tersebut adalah false
15. Output Chakra
16. Output Stamina
17. Jika chakra >= 200, maka hasilnya false
18. Dan Outputnya (“Status: Naruto kehabisan stamina sebelum berhasil.”)
19. Selesai

20. Mulai
21. Mengecek Apakah true
22. Input NIM
23. Jika Input NIM adalah huruf
24. Maka akan mengeluarkan pesan Output/Print (“Input tidak valid! Harap masukkan angka saja.”)
25. Maka akan kembali ke Nomor 2
26. Jika Input adalah angka
27. Maka akan masuk ke kondisi kedua
28. Tinggi_Menara (nim[-2:]), yang di ambil nim saya bagian (2 digit di belakang)¹⁷
29. Gulungan = 0
30. For i in range(3, tinggi_menara + 1, 3)
31. Maka Gulungan += 1, maksudnya gulungan (0 + 1 = 1),
32. Maka akan kembali ke 30, sampai terpenuhi sama fornya (3, 6, 9, 12, dan 15 (maka ada 5 kali putaran, maka gulungannya ada 5)),
33. Output Tinggi_Menara
34. Output Gulungan
35. Selesai
36. Mulai
37. Mengecek Apakah true
38. Input NIM
39. Jika Input NIM adalah huruf
40. Maka akan mengeluarkan pesan Output/Print (“Input tidak valid! Harap masukkan angka saja.”)
41. Maka akan kembali ke Nomor 2
42. Jika Input adalah angka
43. Maka akan masuk ke kondisi ketiga
44. Jumlah_Koridor (nim[-2]), yang di ambil nim saya bagian (1 digit di belakang, sebelum digit terakhir) adalah 1
45. Intelijen = 0
46. Perangkap = 0
47. Jika jumlah_koridor == 1, (dan hasil true) maka
48. For ruangan in range (1, 4), yang di ambil index 1-3
49. Jumlah_Ruangan = Ruangan
50. If Jumlah_Ruang %2 == 1, maka kalau ganjil intelijen tambah 1, Jika genap maka perangkap tambah 1, dan akan Kembali ke Nomor 48, sampai 3 kali putaran
51. Output Jumlah_Koridor
52. Output Jumlah_Ruang
53. Output Intelijen
54. Output Perangkap

55. Jika hasil nya false, maka
56. Jika Jumlah_Koridor > 1
57. For ruangan in range (1, Jumlah_Koridor * 3 + 1), (misalnya 1-4, yang di ambil index 1-3)
58. Jumlah_Ruangan = Ruangan
59. If Jumlah_Ruang %2 == 1, maka kalua ganjil intelijen tambah 1, Jika genap maka perangkap tambah 1, dan akan Kembali ke Nomor 57, sampai (tergantung hasil (Jumlah_Koridor * 3 + 1)) kali putaran
60. Output Jumlah_Koridor
61. Output Jumlah_Ruang
62. Output Intelijen
63. Output Perangkap
64. Selesai

2. Deskripsi Singkat Program

2.1. Tujuan Program

Program ini dibuat untuk **mengolah data NIM (Nomor Induk Mahasiswa)** yang dimasukkan oleh pengguna, lalu **menjalankan tiga misi simulasi bertema Naruto** berdasarkan angka-angka terakhir dari NIM tersebut. Program ini melatih pemahaman logika perulangan (while, for) dan percabangan (if-else) dalam Python.

2.2. Fungsi dan Manfaat Utama Program

- Validasi Input:
Program memastikan bahwa NIM yang dimasukkan berisi angka saja. Jika pengguna salah memasukkan (misalnya huruf), program akan meminta input ulang.
Manfaat: Melatih cara membuat validasi data agar program tidak error.
- Misi 1 – Rasengan:
Menghitung proses pengumpulan *chakra* dan pengurangan *stamina* sampai batas tertentu.
Manfaat: Melatih penggunaan perulangan `while` dan logika kondisi dalam simulasi proses yang berulang.
- Misi 2 – Infiltrasi Menara:
Menghitung jumlah *gulungan informasi* berdasarkan tinggi menara yang ditentukan dari dua digit terakhir NIM.
Manfaat: Melatih perulangan `for` dengan langkah tertentu dan penggunaan operasi aritmetika.
- Misi 3 – Penyelidikan Markas:
Menghitung berapa banyak *data intelijen* dan *perangkap peledak* yang ditemukan berdasarkan jumlah koridor (diambil dari satu digit terakhir NIM).
Manfaat: Melatih logika pengulangan bersarang dan pengambilan keputusan dengan kondisi berbeda.

3. Source Code

```
while chakra < 200 and stamina > 0:
    chakra += 5
    stamina -= 3

print("\n=== Misi 1: Rasengan ===")
print(f"Chakra terkumpul: {chakra}")
print(f"Sisa stamina: {stamina}")
if chakra >= 200:
    print("Status: Naruto berhasil menyempurnakan Rasengan!")
else:
    print("Status: Naruto kehabisan stamina sebelum berhasil.")
```

```
for i in range(3, tinggi_menara + 1, 3):
    gulungan += 1

print("\n=== Misi 2: Infiltrasi Menara ===")
print(f"Tinggi Menara: {tinggi_menara} meter")
print(f"Gulungan informasi yang dikumpulkan: {gulungan}")
```

```
if jumlah_koridor == 1:
    for ruangan in range(1, 4):
        jumlah_ruang = ruangan
        if jumlah_ruang % 2 == 1:
            intelijen += 1
        else:
            perangkap += 1

    print("\n=== Misi 3: Penyelidikan Markas ===")
    print(f"Jumlah koridor: {jumlah_koridor}")
    print(f"Jumlah Ruangan: {jumlah_ruang}")
    print(f>Data Intelijen ditemukan: {intelijen}")
    print(f"Perangkap Peledak dijinakkan: {perangkap}")
elif jumlah_koridor > 1:
    for ruangan in range(1, jumlah_koridor * 3 + 1):
        jumlah_ruang = ruangan
        if jumlah_ruang % 2 == 1:
            intelijen += 1
        else:
            perangkap += 1
    print("\n=== Misi 3: Penyelidikan Markas ===")
    print(f"Jumlah koridor: {jumlah_koridor}")
    print(f"Jumlah Ruangan: {jumlah_ruang}")
    print(f>Data Intelijen ditemukan: {intelijen}")
    print(f"Perangkap Peledak dijinakkan: {perangkap}")
else:
```

```
print ("Angka adalah 0, atau angka tersebut minus")
```

4. Hasil Output

```
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\pratikum apd\post-test\post-test-apd-4> & C:/Us
/APD/Pratikum/pratikum apd/post-test/post-test-apd-4/2509106117-Muhammad_Fajar-PT-4.py"
Masukkan NIM Anda: 2509106117

=== Misi 1: Rasengan ===
Chakra terkumpul: 195
Sisa stamina: 0
Status: Naruto kehabisan stamina sebelum berhasil.

=== Misi 2: Infiltrasi Menara ===
Tinggi Menara: 17 meter
Gulungan informasi yang dikumpulkan: 5

=== Misi 3: Penyelidikan Markas ===
Jumlah koridor: 1
Jumlah Ruangan: 3
Data Intelijen ditemukan: 2
Perangkap Peledak dijinakkan: 1
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\pratikum apd\post-test\post-test-apd-4> |
```

Gambar 4.1 Output

5. Langkah-Langkah Git

5.1. Git Add .

```
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\pratikum apd> git add .
```

Gambar 5.1.1 Git Add .

Perintah git add . digunakan untuk menambahkan semua perubahan file yang ada di dalam folder proyek ke dalam staging area Git. Staging area adalah tempat sementara di mana perubahan file disiapkan sebelum benar-benar disimpan ke dalam riwayat repository melalui perintah git commit.

Tanda titik (.) setelah git add berarti semua file yang ada di direktori saat ini, termasuk subfolder di dalamnya, akan ikut ditambahkan. Jadi, jika ada file baru, file yang sudah diubah, atau file yang dihapus, semuanya akan masuk ke dalam staging area sekaligus.

Contohnya, ketika kita selesai mengedit beberapa file sekaligus, daripada menambahkan file satu per satu dengan `git add namafile`, kita bisa langsung mengetik `git add .` agar semua perubahan tercatat dalam satu langkah. Setelah itu, barulah perubahan tersebut bisa disimpan secara permanen menggunakan `git commit`.

Namun, karena `git add .` menambahkan seluruh perubahan, kita perlu berhati-hati agar tidak secara tidak sengaja menyertakan file yang sebenarnya tidak ingin dimasukkan ke dalam repository. Untuk menghindari hal ini, biasanya digunakan file `.gitignore` agar Git mengabaikan file tertentu.

5.2. Git Commit

```
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\pratikum apd> git commit -m "Menambahkan Folder poster-apd-4"
[main 4f2f706] Menambahkan Folder poster-apd-4
1 file changed, 8 insertions(+), 4 deletions(-)
```

Gambar 5.2.1 Git Commit

Commit dalam Git dapat diibaratkan seperti menyimpan catatan atau rekaman atas perubahan yang telah dilakukan pada proyek. Setiap kali kita melakukan commit, Git akan menyimpan kondisi file yang sudah dimasukkan ke staging area sebagai satu titik riwayat baru. Titik riwayat ini nantinya bisa dilihat, dilacak, bahkan dikembalikan lagi jika dibutuhkan.

Sebuah commit biasanya disertai dengan pesan (commit message) yang menjelaskan apa perubahan yang dilakukan. Misalnya, ketika menambahkan fitur baru, memperbaiki bug, atau mengubah bagian tertentu dari kode. Pesan commit ini sangat penting agar orang lain — atau bahkan diri kita sendiri di kemudian hari — dapat memahami tujuan dari perubahan yang dibuat.

Setiap commit memiliki identitas unik berupa hash (serangkaian angka dan huruf) yang membuat Git bisa membedakan satu commit dengan commit lainnya. Dengan adanya commit, kita bisa menelusuri riwayat proyek dari awal hingga kondisi terbaru, serta berkolaborasi dengan lebih teratur.

5.3. Git Push

```
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\pratikum apd> git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 582 bytes | 582.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Celrik08/praktikum-apd.git
b27686c..4f2f706 main -> main
```

Gambar 5.3.1 Git Push

git push adalah perintah yang digunakan untuk mengirimkan atau mengunggah commit dari repository lokal ke repository remote, misalnya ke GitHub, GitLab, atau Bitbucket. Dengan melakukan push, semua perubahan yang sudah kita simpan melalui commit di komputer akan tersalin ke repository yang ada di server sehingga bisa diakses oleh orang lain atau digunakan pada perangkat lain.

Pada contoh di atas, origin adalah nama remote (default ketika kita menambahkan repository GitHub), dan main adalah nama branch yang akan dikirim. Artinya, semua commit yang ada di branch main pada komputer kita akan diunggah ke branch main di repository remote.

Tambahan -u (atau --set-upstream) berfungsi agar pada push berikutnya kita cukup mengetik git push tanpa harus menuliskan nama remote dan branch lagi, karena Git sudah mengingat pengaturan tersebut.

Dengan kata lain, git push adalah cara kita membagikan hasil kerja dari repository lokal ke repository online, sehingga proyek bisa ter-update dan mudah dikelola bersama tim.